CRLF Injection Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ CRLF Injection — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in an authorized lab or in-scope application. Use harmless redirect destinations and avoid injecting headers that affect real users.*

■ Executive Summary

CRLF injection occurs when carriage-return and line-feed characters from untrusted input are allowed to alter the structure of an HTTP header or response. The supplied one-page PDF focuses on URL parameters associated with redirects and mentions encoded representations that may be decoded differently by application layers.

■ Core Testing Methodology

1. **Find header-influencing inputs.** Review parameters that affect redirects, language selection, cookies, or response headers.
2. **Record baseline behavior.** Capture the normal request and response for a controlled test value.
3. **Review decoding.** In a lab, check whether the application decodes input once or multiple times and whether validation happens before or after canonicalization.
4. **Check response construction.** Verify that untrusted values cannot create additional response headers or split a response.
5. **Check redirect validation.** Confirm that redirect destinations are validated against an allowlist or constrained to safe relative paths.
6. **Inspect downstream behavior.** Verify that proxies, caches, and application servers handle invalid control characters consistently.
7. **Document evidence.** Record the parameter, response header behavior, and safe impact demonstration.
8. **Retest the fix.** Confirm control characters are rejected or safely encoded.

■ High-Value Bug Bounty Tips & Edge Cases

- **Redirect parameters:** The source recommends looking at parameters that lead to redirects and inspecting 3xx responses.
- **Encoded characters:** Double-encoded and alternate-encoding representations can expose inconsistent decoding. Use inert test values in a lab rather than redirecting users to hostile sites.
- **Response headers:** Inspect `Location` and `Set-Cookie` behavior for unexpected line breaks or additional headers.
- **Multiple layers:** A proxy and application may decode or normalize values differently.
- **Source limitation:** The source is a one-page summary and does not establish a vulnerability in any particular application.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Parameter: [Redirect-or-Header-Related-Parameter]
Test value: [Harmless synthetic marker]
Expected: [Control characters rejected or safely encoded]
Observed response headers: [Redacted]
Unexpected header created: [Yes/No]
Evidence: [Redacted]
```

- [ ] Untrusted input cannot create response headers.
- [ ] Redirect targets are validated.

- [ ] Canonicalization occurs consistently.
- [ ] Invalid control characters are rejected.
- [ ] Proxies and application servers use consistent parsing.

■■ Mitigation & Remediation

- Reject CR and LF characters in values used to construct headers.
- Use framework APIs for response headers rather than manual string concatenation.
- Validate redirect destinations with allowlists or safe relative paths.
- Canonicalize input before validation and avoid repeated decoding.
- Keep reverse proxies, caches, and application servers patched.
- Add regression tests for encoded and malformed control characters.

■ Source Notes

Based on the supplied one-page *CRLF Injection.pdf*. It defines the issue, recommends reviewing redirect-related parameters and 3xx responses, and shows basic, double-encoded, and alternate-encoding examples. This rewrite preserves those concepts without reproducing redirect-to-hostile-site payloads.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.